Jinhee Mok

jinheemok815@gmail.com
jin-tonix.github.io · GitHub
Gangnam-gu, Seoul

AI & Automation Engineer · From design to deployment & operations

Career Description

I majored in law and worked as a daycare operations manager and an English teacher. In 2024 I learned development at Himedia Academy, and as a We:Review intern I automated receipt review with OCR and an LLM. I now move repetitive field work into AI agents and automation. I design systems to stop at candidates and proposals, with staff making the final call. I hand much of the coding to an AI coding tool (Claude Code); deciding what to build, what to turn on in production, and checking the results is my job.

STYLESELLER · PL (AI & Automation) · 2025.11 ~ Present

Moved vendor record-keeping, responses, and calculations that staff repeated by hand into AI agents and an internal work tool, and own deployment and operations.

1. Status Board · Operations Ledger

Built 2026.10.19 ~ 10.23, in production

Role: Designed the ledger schema and counterpart-linking rules; built the intake API, the mail collector, and 6 screen tabs; ran DB tests and verified the production screens.

What I built: Summarizes KakaoTalk rooms, the order mailbox, Slack, and Excel order sheets attached to emails on a daily basis and places them in date order on each counterpart's calendar. The same record showing up more than once still leaves only one entry in the ledger. Records are linked to counterparts without an LLM, by a fixed rule order (thread, business registration number, KakaoTalk room, email, etc.); if the keys conflict, the record is left unclassified for staff to resolve.

Results: Shows the deal stage of 352 counterparts (147 brands, 135 vendors, 70 sellers) on one screen (2026-10-24). Collects roughly 100-200 records per weekday; records the rules cannot link automatically are connected by staff on the counterpart-check screen (hundreds pending as of 10-24). Rewrites summaries for up to 300 counterparts every day at 18:05, and stops that day's batch if the daily cost exceeds KRW 2,000.

Tech: React, Tailwind, Vitest, Python 3.12, Pydantic v2, Postgres RLS, Gemini Flash family (via LiteLLM)

2. Content Autopilot

2026.10.03 ~ Present

Role: Designed the stage state machine and the publishing defaults; built the orchestrator, the publish gate and DB slots, and the QA gate; ran the production canary and restart-scenario tests.

What I built: When a staff member requests a Reel in the work tool, it writes the script, renders a 9:16 MP4, and gets approval on a preview. A state machine with no LLM drives stage progression. Defaults are manual publishing, dry-run for recurring automation, and paid generation off, and the "AI-generated content" label is added automatically.

Results: The production canary has been verified up through saving a 1080×1920 MP4 to Drive (2026-10). The engineering for this new production path is done, but details such as prompts are still being tuned, so there have been no real SNS posts through it yet; a separate, already-live path swaps only the caption/hashtags per account and publishes existing videos to multiple accounts simultaneously. One post per account per day is enforced by a DB index. 617 test files across the 3 services.

Tech: content orchestrator, Hermes, Gemini Flash family (via LiteLLM), video renderer, HyperFrames, ffprobe, Google Drive, Supabase (PostgreSQL)

3. Vendor CS Agent

2026.06 ~ Present

Role: Designed the collection path, the vendor gate, and the evidence-matching rules; built the server agent, send/delivery confirmation on the kakaocli fork, and the Windows collector; handled incident response and operations.

What I built: Staff PCs collect KakaoTalk conversations and upload them to the server, and replies are also sent from staff Macs. The room name is checked against the list of vendors being served, and the agent does not answer if it cannot identify the vendor. For tagged questions, it searches the vendor's records and the knowledge hub to answer, and checks the amounts, percentages, and brand names in the draft against the evidence in code; if they do not match, it searches again and rewrites the draft.

Results: Cleared a resend bug where a single message arrived 51 times, with 15-second deduplication. Fixed a macOS Accessibility freeze that made 31 of 147 sends (21%) fail, using a 3-second timeout and delivery confirmation (comparing

4. Agent Platform · Spark 2026.05 ~ Present

Role: Designed the work-order, delegation-guard, and tool-gate structure; connected the shared tool/skill box, one-time approval, the LiteLLM cost cap, and the Jev connection; wrote the evaluation set, fixed measurement errors, and deployed to the VM.

What I built: An internal agent platform forked from Paperclip. Staff use Spark to ask read-only questions of 4 business DBs, and hand off work to the CEO agent. Delegation depth is capped at 2 and each work order allows 5 delegations; tool approval is enforced by the server and the DB, not by prompts. BM25 narrows candidates from 115 shared skills, Jev re-filters them by gate and fit score, and Gemini makes the final pick of one skill.

Results: As of 10-24, out of 109 shared skills, the recall of BM25-shortlisted candidates containing the correct answer was 85.5% at rank 1 and 91.3% in the top 3 (69 questions with known answers, 2026-10-24). Each approval executes exactly once (10 concurrent requests, 1 execution, confirmed by tests). Paid calls are capped at KRW 30,000 per day; once the cap is hit, the server returns 403 instead of letting callers retry. Follow-up (2026-10-25~): adding an LLM proposer to the Jev router on the Hermes agent produced 90.15% accuracy (119 of 132, with 115 skills) on new questions reused afterward. Fake skills that existed only in the test folder and answer-key errors were found and corrected, and the numbers were re-measured. The earlier 85.5%/91.3% figures are candidate recall and are not compared against this number. Having BM25 shortlist and Jev re-filter also scored 90.15% (119 of 132) on the same 132 cases, matching Gemini alone, and because Gemini reads far less, cost is estimated at roughly 1/10. This approach is now fully in production and is what the Hermes agent uses to pick a skill.

Tech: Paperclip fork (Express 5, Drizzle, PostgreSQL), MCP SDK, Hermes, Python 3.12, FastAPI, LiteLLM, Redis, Langfuse, Gemini Flash family (via LiteLLM)

5. LightRAG Knowledge Hub · Email Agent 2026.03.31 ~ Present

Role: Designed the workspace registry, the search-planning and evidence-judging steps, and the draft-verification order; built the email-agent server, the checker, and the sent-mail learning cursor; handled deployment and incident recovery.

What I built: A knowledge hub shared by Vendor CS, Influencer CS, Spark, and sales-partner matching. For inquiries to the shared mailbox, it places a reply draft backed by evidence into the person-in-charge's Gmail drafts folder, and staff send it. It checks up to 6 claims in a draft against the evidence; claims with no evidence are rewritten, and if unsupported claims remain after the rewrite, the draft is discarded.

Results: 4 agents share 13 workspaces; the server refuses to start if a name is mismatched. Fixed a bug where 48 of 1,943 sent emails were left out of learning, by changing how the cursor is saved.

Tech: Python 3.12, FastAPI, Pydantic v2, LightRAG (Postgres/pgvector + Neo4j), FlashRank, Gemini Flash family (via LiteLLM), Gmail, pytest

6. Internal Work Tool · Pricing, Matching, Personalized KakaoTalk Late 2026.03 ~ Present

Role: Designed the price-calculation structure, matching-candidate rules, and send-approval criteria; built and operate the pricing code, catalog extraction, matching, send automation, and staff-PC workers. Seller-recommendation logic is owned by a colleague on CRM.

What I built: Moved the Google Sheets pricing formulas into code, down to the rounding units. The LLM only returns the input values to change, as JSON, and code recalculates the amounts. For catalogs (PDF, image, Excel, URL), the LLM extracts product candidates and only items staff confirm are registered. Personalized KakaoTalk messages go out to the recipients received from CRM, and sends of 500 or more wait in an approval queue.

Results: 43 price-calculation test cases, some checked field-by-field against real products' sheet values. Sent personalized KakaoTalk messages to 261 people in one day (2026-09-22). Seller-search workers run on 9 of 11 staff PCs, with the shared key replaced by a per-PC key (2026-09).

Tech: React, Vitest, Postgres+PostgREST, Gemini Flash family (via LiteLLM), knowledge-hub RAG search, kakaocli

7. Staff PC Deployment · Operations 2026.02 ~ Present

Role: Designed the install/auth/update structure and the server-side reference date; built the installer, the worker, the Windows collector, and changes to the kakaocli fork; handled deployment, incident response, and monitoring cron jobs.

What I built: Runs KakaoTalk and Instagram tasks on staff Macs and Windows PCs. Install is a one-line command; authentication issues a per-device key via a one-time token; updates run on an automatic schedule staggered per PC. The server deploys only from committed images and halts the deploy if an image is missing.

Results: One-line install deployed to 11 staff PCs (Mac and Windows) (as of 2026-09-16 registration). When a cloud DB usage

WE:REVIEW · AI Development Intern · 2025.08 ~ 2025.10

Built a review pipeline that extracts products and prices from receipt images as JSON: PaddleOCR tile splitting, DBSCAN line reconstruction, few-shot prompting on an open LLM (Qwen family), served via FastAPI. Raised review accuracy from 70% to 95% and cut review time by 85% (measured directly).

Previous Experience

English Teacher · Jeongsang Language School (2018.08 ~ 2020.09), Pots&Pans (2021.03 ~ 2022.05), Montekids English Academy (2022.06 ~ 2024.02): lesson design, teaching materials, parent consultations.

Jeonwon Daycare Center & After-School Program · Operations Manager (2015.03 ~ 2018.02): teacher hiring, curriculum & event planning, running the center on behalf of the director.